Individual Report

MiniOS Alpha | Operating Systems Final Project

Student Name:

Group: Marquez and Friends

Assigned Role: Memory Management

What did I contribute to the project?

I built the memory manager in memory.py. It simulates 1024 MB of memory and lets the user see the status, allocate memory and free it. I also wrote the two methods that let the process manager reserve and release memory, so running programs use up memory like they would in a real system.

What features did I personally develop?

- A MemoryManager with 1024 MB total and 256 MB used by the system at start-up
- memory, allocate and free (the required commands)
- Three separate counters: system memory, user-allocated memory and process memory
- Limit checks for every request
- reserve_for_process and release_from_process, used by Student 3's process manager
- The Memory Status screen in the format shown in the project sheet

What problems did I encounter?

- A user running free must not be able to release memory that belongs to the system or to a running process.
- Requests that are zero, negative or larger than the free memory.
- The Memory Status numbers had to line up in neat columns.
- Keeping memory correct when processes are started and killed.
- Agreeing with Student 3 on how the two modules talk to each other.

How did I solve those problems?

- I split memory into three counters. free only touches the user counter and refuses amounts larger than what the user allocated.
- allocate and free check that the amount is above zero, and allocate also checks it against the free memory and says how much is actually free.
- I used fixed-width formatting so the numbers line up under each other.
- Used memory is always calculated from the three counters, so it cannot drift out of step.

- We agreed on reserve_for_process and release_from_process as the only way the process manager changes memory.

What did I learn from this project?

I learned that memory management is about limits and accounting: every request has to be checked against what is free, and the system has to know who is using what. Keeping separate counters made it easy to protect system and process memory from the user commands.

Evidence of my contribution:

- memory.py (MemoryManager)
- The commands memory, allocate and free in main.py

Sample output from my part of the program:

```
MiniOS \> memory
Memory Status
------------------------
Total Memory:  1024 MB
Used Memory:    256 MB
Free Memory:    768 MB

MiniOS \> allocate 200
200 MB allocated.

MiniOS \> allocate 5000
Error: Not enough free memory. Only 568 MB available.

MiniOS \> free 999
Error: You only have 200 MB allocated to free.
```

Screenshots (attach in the screenshots folder and list the file names here):